

INTERNAL BRIEFING · AUGUST 14, 2026



NaturalLens

Grant interview briefing.
Tech facts you can say out loud.

Sixty-second pitch

NaturaLens is a phone app that names the animal in front of you and keeps a log of what you found. Identification today is a hosted vision-language model. In parallel we are labeling our own field photos and fine-tuning a small detector so the same loop can later run on-device, offline, and cheap at scale.

PRODUCT LOOP	IDENTIFY NOW	LATENCY
Capture → identify → save	Gemini 3.1 Flash-Lite	~1.5–2 seconds
APP	HOSTING	CUSTOM MODEL
Expo + EAS / TestFlight	Cloudflare Workers + D1	SSDLite, polar bear
TRAINING STYLE	LABELER	FINDS STORAGE
Transfer learning, not scratch	Live at skim.naturalens.ca	On the phone only

Never claim: we trained a foundation model · we fine-tune CLIP/ResNet in the shipped app · identification is on-device today · we are on the public App Store · we have accounts, maps, or GPS · we store user sightings on our servers · we licensed a photo library · we run AWS/GCP app servers · the custom detector is in the phone yet.

Say this instead

- Cloud foundation-model ID now, so we can test the product in the field.
- Fine-tune a compact pretrained detector (SSDLite / YOLO-class) once labels exist.
- Grant money runs Gemini quota, field photos, and the dataset — not a GPU cluster, and not the Apple fee (already paid).
- Polar bear is the first class because it is tractable and conservation-relevant.

Stage, honestly

- Apple Developer is enrolled. iOS alpha path is EAS → TestFlight — not a public App Store listing.
- Landing + waitlist live on naturalens.ca.
- Internal polar-bear labeler + training loop exist.
- On-device inference is the destination, not the current binary.

What is built vs not

PIECE	STATUS	WHAT TO SAY
Mobile app	Shipped (alpha)	Expo SDK 54, TypeScript. Camera → Gemini → local finds list. Survives restart. Detail view + delete.
Identification	Live	Google Gemini API, model gemini-3.1-flash-lite. Photo resized to 1024px. Structured JSON back.
Website / waitlist	Live	Next.js on Cloudflare Workers. D1 table of emails. Domain naturalens.ca / www.
Labeler (Skim)	Live, internal	Upload, draw boxes, review, create immutable training runs. R2 images + D1 labels. PIN-gated.
Custom detector training	Code live; weights not in the app	SSDLite320 MobileNetV3-Large, one-class polar bear, COCO-pretrained transfer, Colab T4 or local GPU. Exports ONNX / TFLite.
Our backend API	Not started	Placeholder. Next job: proxy Gemini so the key leaves the phone.
Inference microservice	Not started	Not needed until we serve our own weights. Phone talks to Gemini directly today.
GBIF / IUCN species DB	Not started	Planned enrichment. Conservation status today comes from Gemini's schema enum, not a database.
Accounts, sync, map, GPS	Not in MVP	History is local. Location is a later feature and cannot be backfilled onto old finds.
iOS distribution	Account ready	Apple Developer enrolled. We can EAS-build and send testers a TestFlight link. That is closed alpha, not a public App Store listing. A TestFlight build is not in testers' hands yet — the account unblocks it.
Google Play	Not yet	No Play Console account yet. Android testers still use Expo Go or a local/dev build until the \$25 enrollment.
On-device TFLite in the app	Not wired	EAS can now ship a custom native binary (the Expo Go module limit is no longer the blocker). TFLite still is not in the app. Wire it once quality + latency bars are met.

What we actually run

LAYER	CHOICE	WHY
Mobile	React Native via Expo SDK 54, React 19, TypeScript. EAS for iOS builds.	One codebase, camera APIs. Apple Developer enrolled → TestFlight via EAS. Expo Go remains for local iteration.
Camera / images	expo-camera, expo-image-manipulator, expo-file-system	Capture, shrink to 1024px for ID, persist display + thumb copies.
Local state	React context + AsyncStorage	Right size. No Redux. History key is versioned (naturalens-history-v1).
Web	Next.js 16, Cloudflare Workers, D1	Landing + waitlist. No Kubernetes.
Labeler	Cloudflare Worker, R2, D1, static UI	Internal tool. Free-tier caps enforced at 80%.
Training	PyTorch + torchvision SSDLite, Colab T4 or Apple MPS	Apache/BSD via torchvision. EfficientDet-Lite blocked on TF <2.16 / Python 3.12.
Repo	Single monorepo	Small team, atomic PRs, one clone.
License	MIT	Code is MIT. Model weights and field photos are not committed.

Two systems, one product

Interviewers often hear “AI wildlife app” and assume one model. We have a product path and a model path. Keep them separate when you talk.

A. What the user touches (alpha)

Phone camera → freeze the still → resize 1024px JPEG → Gemini Flash-Lite (JSON schema) → species + confidence + facts → user saves → AsyncStorage metadata + two JPEGs in the app document directory.

There is no NaturaLens server in this loop. The phone calls Google. Finds never leave the device except the photo bytes sent for that identify call.

B. What we train in parallel (internal)

Field photo → Skim upload (EXIF-normalized, sha256 id) → R2 + D1 boxes → Create run (immutable manifest) → Colab SSDLite train → metrics / preds / TFLite back to R2 → review failures → relabel.

This loop is polar-bear detection (bounding box), not yet the multi-species namer in the phone. Gemini still answers “what species is this?” for testers.

Why cloud ID now

We can leave Expo Go now: Apple Developer + EAS produces a real iOS binary (and can include custom native modules later). Cloud ID is still the right call because the custom detector is not wired, the dataset is not at the switch bar, and we can iterate the product loop without freezing a model.

Why on-device later

Cloud on 4G is typically 2–9s, not <1s. It needs signal, costs per call, and sends photos off-device. A field tool that dies without LTE is a compromised field tool. On-device + cloud fallback is the destination.

Gemini call — facts, not vibes

DECISION	VALUE	MEASURED REASON
Model	gemini-3.1-flash-lite	~1.5s vs ~7–18s for Flash, same answers on test images. Naming an animal does not need a reasoning model.
Image size	1024px wide JPEG, quality 0.8	Gemini tiles images. 512px saved nothing on tokens or latency; it only lost detail.
Thinking budget	minimal	Default thinking burned ~230 reasoning tokens of pure latency.
Output	JSON schema	isAnimal, label, confidence, description, habitat, diet, conservationStatus
Extra fields cost	~230ms / 16%	Median 1454ms → 1681ms vs a 3-field schema. Worth it vs a second round-trip.
Non-animal	isAnimal: false → empty list	Without this guard the model will happily label a chair as a species at confidence 1.
Confidence	Self-reported, clamped to [0,1]	Not a calibrated probability. Treat as a hint. Do not quote it as accuracy.

Species facts ride the same call. Older saved finds without facts are backfilled with a text-only lookup when opened — facts follow from the label, not the pixels. Location could never be backfilled that way, which is why we have not added GPS yet without a plan.

Are you training a model?

Short answer: The app testers use is not running our weights. We are fine-tuning a small pretrained detector on our polar-bear photos. We are not training from scratch and we are not fine-tuning CLIP or ResNet for the product.

Architecture

ITEM	CHOICE
Family	One-stage object detector, not a classifier-only head.
Network	SSDLite320 + MobileNetV3-Large (torchvision).
Classes	2: background + polar_bear.
Init	Transfer matching tensors from COCO SSDLite weights. Not random init.
Optim	AdamW, lr 1e-3, weight decay 1e-4, cosine annealing, grad clip 10.
Default run	30 epochs, batch 4, seed 13.
Augment	Horizontal flip, scale-and-pad, brightness/contrast.
Metric	mAP@0.5, plus precision/recall at score 0.3. Not “accuracy.”
Export	PyTorch checkpoint, ONNX, float16 TFLite when conversion works.
Compute	Google Colab T4, or local Apple MPS / CUDA. No standing GPU cluster.

EfficientDet-Lite via MediaPipe Model Maker is the planned mobile fallback, blocked today because it wants TensorFlow <2.16, which has no Python 3.12 wheels. Say “SSDLite now, EfficientDet-class if we need a smaller mobile graph.”

Data scale we will quote if asked for a number

Internal architecture notes (targets, not a claim we already have this):

PROBLEM	MINIMUM	BETTER
Single species detector (e.g. polar bear / deer)	1k–2k labeled images	3k–8k
Two similar species (polar vs brown bear)	1.5k–3k per class	5k+ per class
Multi-species (10–20)	1k+ per species	3k–10k per species
Hard negatives (no animal / confusers)	30–50% of samples	

Early packs were on the order of tens of photos (e.g. 44/20 train/val) — enough to stand up the loop, not enough to claim production mAP. If they ask “how much data do you have?”, say: the pipeline is live; the dataset is still small; the grant is partly to grow labeled field photos, not to buy stock.

How labels get into the model

Store	Integrity	Cost control
• Images: Cloudflare R2 bucket. • Boxes / review flags / run metadata: D1. • Primary key is sha256 of the normalized JPEG — duplicates do not fork the dataset.	• Optimistic version → 409 on conflict. • Append-only label history. • EXIF orientation fixed once at upload. • Training run copies an immutable manifest. You cannot silently edit the set mid-train.	• Hard caps at 80% of R2 free tier (8 GB, 800k Class A, 8M Class B). • Max object 20 MB. Over cap → HTTP 429. • Colab reports S3 ops so Worker + GPU share one meter.

Photos are ours (field captures), not licensed stock. Later species facts: GBIF / IUCN open data. Do not budget photo-library licenses.

Where the bits go

DATA	WHERE IT LIVES	LEAVE-THIS-OUT
Identify photo	Sent to Google Gemini for that request. Resized; not our archive.	Do not say “photos never leave the phone.” They leave for ID.
Saved finds	On device: JSON in AsyncStorage, JPEGs in the app document directory (not camera cache).	Do not say we have a sightings database.
Waitlist email	Cloudflare D1. Deduped. Stores user-agent + IP at signup.	Fine to say we collect email for early access.
Labeler photos	Our R2, internal PIN auth. Not user production data.	Do not describe it as a public citizen-science upload portal.
GPS	Not in the MVP. Deliberate.	Privacy page mentions location as a future option. The app does not collect it yet.
Gemini API key	EXP0_PUBLIC_* is inlined in the client bundle.	Restricted in AI Studio. Next engineering step is a Worker proxy. TestFlight is a closed tester list — still do not share a public App Store / unrestricted link until the key is proxied.

If they ask about privacy: identification requires a network call to Google today. The log is local. We do not operate user accounts. Moving inference on-device is how we get both privacy and offline, together.

If they ask about IP: application code is MIT. We do not claim ownership of Gemini. Our labeled boxes and future weights are our dataset/model artifacts; weights are gitignored. torchvision SSDLite is Apache/BSD.

Questions they actually ask

What problem are you solving?

Field identification is either slow (guides, experts) or locked to apps that need an account, a community, or a live model download you cannot iterate. We want a fast personal loop: point, name, keep. Conservation and education only work if the tool is used in the places with bad signal — which is why on-device is the end state, not a slogan.

Who is the user?

Near term: closed testers and field partners who will photograph wildlife and correct us. Polar bear is the first detection class because the team can collect and label it. Consumer “what bird is that?” is the same loop at larger taxonomy later.

How do testers install the app?

iOS: Apple Developer is enrolled. We build with Expo EAS and invite testers on TestFlight — a closed list, not the public App Store. Android: Play Console is not enrolled yet; those testers still use Expo Go or a local build. Do not say the app is “on the App Store.”

Why not just use iNaturalist / Seek / Google Lens?

Seek is the existence proof that on-device vision plus geo ranking works — we say that in our own notes. We are earlier: a simpler personal log, our own labeled field data, and a path to region packs (species priors by location/season). We are not claiming we beat Seek’s model today. We are building the capture → label → train loop we would need to own the model instead of renting Gemini forever.

Is this scientifically valid identification?

No. It is an assistive namer with self-reported confidence, not a taxonomic authority. Conservation status is an enum the model fills; we fall back to Data Deficient rather than invent a Red List row. A future GBIF/IUCN join is how we stop asking a language model to be a database.

What is novel here technically?

Do not oversell novelty. The novel-for-us work is the closed loop: product in the field, our own boxed photos, immutable train runs, failure review, export to TFLite. The architectures (Flash-Lite, SSDLite) are known. Grants fund execution and data, not a new paper architecture.

Why polar bears first?

One class, visually distinct in many frames, conservation story, and a dataset we can actually label. Similar-species pairs (polar vs brown) are a later, harder milestone and need more images per class. Starting multi-species at 10k classes is how you never ship.

How will you know the custom model is good enough to replace Gemini?

Not a single accuracy number. All of: precision/recall per critical class, stable performance in hard conditions, on-device latency under one second on target phones, then a staged rollout (10% → 50% → 100%) with Gemini still as fallback when confidence is low.

What if Google changes pricing or shuts the model?

That is exactly why we are not building the company on a forever Gemini bill. Alpha uses it as a scaffold. The labeler and SSDLite export are the off-ramp. Short term we put a spend cap on the Google project and treat HTTP 429 as a normal user-facing “slow down” state — free-tier limits are already tight.

Why Cloudflare instead of AWS?

The surfaces we have today (site, waitlist, labeler, object store) fit Workers + D1 + R2 at essentially free-tier cost. We do not have a GPU cluster or a multi-service VPC to justify AWS. Training uses Colab or a laptop GPU. We will rent GPU hours when a run needs them — not run idle instances for a grant report.

How do you handle bad or malicious labels?

Internal tool, allowlisted emails, PIN + lockout. Versioned boxes, append-only history, reviewed flag. Architecture notes also call for an admin queue before user-corrected labels enter training — that user-facing confirm/correct UI is not in the phone yet, so we are not claiming citizen labels as ground truth today.

What’s the biggest technical risk?

Three: (1) Gemini key in the client until we proxy it — TestFlight is still a closed list, not a public link. (2) Dataset too small / too clean, so mAP on our val set lies about field performance. (3) TFLite is not in the binary yet. EAS can ship it; the model has not cleared the switch bar.

Do you have tests / CI?

CI typechecks the mobile app on main. There is no test runner. Say it if asked; do not volunteer it as a strength. Grant engineering time can include a holdout eval harness — we already dump per-image TP/FP/FN for the labeler overlay.

Open source? Will the model be public?

Code is MIT. Field photos and weights are not in git. We have not committed to releasing weights. Do not promise a public model unless the grant requires it — then we can, with license on the weights separate from the app.

How will you measure success in 3–6 months?

Product: testers on TestFlight complete capture → identify → save without 429s; Gemini key proxied off the phone. Data: labeled polar-bear set grown toward the 1k+ target; val mAP tracked per run, not a one-off notebook. Model: TFLite artifact timed on a real phone via an EAS build, even if Gemini is still the default.

What to ask for

Apply. Ask for a small operating budget to run the alpha and grow labeled photos — not a program to pretrain a foundation model. A defensible 3–6 month band is about **\$2,000–\$8,000** (or the grant’s mini-award). Apple Developer is already paid. iOS TestFlight via EAS does not depend on this grant; Gemini quota and field photos still do.

LINE	WHY IT IS REAL	ESTIMATE
Gemini / Google AI, spend-capped	Free tier 429s will stall testers. Flash-Lite is cheap per photo (fractions of a cent: on the order of \$0.25/M input tokens, \$1.50/M output; a 1024px image + short JSON is small).	\$200–500
Apple Developer	Already enrolled. Unlocks EAS → TestFlight. Not a grant line.	Paid — \$0 to ask
Google Play Console	Android internal testing. Still needed for Play testers.	\$25 one-time
Expo EAS	iOS cloud builds + TestFlight submit. Free tier is 30 builds/month; pay only if we outrun it or want faster queues.	\$0–99/mo if we leave free
Domain / Cloudflare overage	naturalens.ca already on Workers. Buffer if traffic grows.	\$20–50
Test device	Simulators have no camera. Mid-range Android if we lack one.	\$150–400
Field travel / capture time	Our dataset plan. Not stock licenses.	\$200–1,000
Worker to proxy Gemini	Engineering, not GPU. Infra ~\$0 at this scale.	time, not cash
Colab / GPU hours	Only when a training run needs more than T4 free. Modest fine-tune, not pretrain.	\$0–300 parked

Do not headline: GPU cluster, AWS, photo-library licenses, paid labeling contractors, “train CLIP from scratch.” If the form wants a bigger number, split ~80% product / testing / API, ~20% dataset prep toward the fine-tune.

Unit economics you can say

Cloud ID cost scales with photos. On-device cost is download-once. That is the business case for the model work: Gemini is the right scaffold and the wrong forever bill. Cloudflare hosting at current scale is effectively \$0. R2/D1/Workers free tiers cover the labeler if we stay under the hard caps we already coded.

Next, later, not now

Next	Later	Not now
• EAS production build → TestFlight to the closed tester list. • Proxy Gemini behind our Worker so the key leaves the client before any wide invite. • Grow polar-bear boxes toward the 1k floor. • User confirm / correct in the app, then an admin review queue.	• Wire TFLite / Core ML into an EAS binary (dev client / production), now that we can ship custom native modules. • Geotag + map. Region “animal packs” (geo prior × vision score). • GBIF / IUCN species database so facts are not model prose. • More classes; similar-species pairs.	• Accounts and cloud sync of finds. • Live video detection. • Training a foundation model. • Standing AWS/GCP inference cluster.

OPS THEY MAY CALL “PRODUCTION-READINESS”

What the user sees when it fails

FAILURE	COPY IN THE APP
Network / fetch reject	You’re offline. Connect and try again.
HTTP 429	Too many photos too quickly — wait a moment and try again.
403, or 400 with “api key” in the body	Your Gemini API key was rejected.
5xx	Gemini is having trouble right now.
Anything else	Couldn’t identify that photo. Try again.

A 400 is not automatically a bad key — a malformed request is also 400. We check the body. Fetch only rejects on network failure; 4xx/5xx still resolve. That is why “you’re offline” is safe in the catch.

CLOSING LINE

If you only remember one paragraph

We already have a working identify-and-log app, a live waitlist, an internal labeling-and-training loop for a polar-bear detector, and an Apple Developer account so we can EAS-build to TestFlight. Identification in testers’ hands is Gemini Flash-Lite. The grant keeps that loop running, funds Gemini quota and field photos, and grows our own labels so we can fine-tune a small on-device model — transfer learning, not a foundation model from scratch.

Sources: repo READMEs, docs/DESIGN.md, docs/Architecture_Notes.md, apps/labeler, models/training, apps/mobile/src/lib/detector.ts. Briefing date 14 August 2026. Do not claim numbers we have not measured in front of them (production mAP, monthly active users, photo count beyond “small / growing”).